

SEIF Project

Self-Evolving Reinforcement Learning for Instruction Following

Self-Evolving Instruction Following Framework

This project implements an iterative self-training pipeline
with two mutually improving roles:

Instructor: Generates diverse multi-constraint instructions

Follower: Learns to follow multi-constraint instructions

1. Project Overview

SEIF (Self-Evolving Instruction Following) is an iterative self-training system based on the Self-IF framework. The core idea is to continuously improve through mutual promotion of two models: in each iteration, the latest Follower model is used as the evaluation/filter/answer model to train the next Instructor, forcing it to generate instructions that the current Follower cannot yet follow well.

1.1 Key Innovations

- * Iterative Self-Training: Continuous evolution through alternating Instructor and Follower training
- * Multi-Constraint Generation: Instructor learns to generate complex instructions with multiple atomic constraints
- * Intelligent Constraint Evaluation: Uses vLLM models for conflict detection and compliance assessment
- * Flexible Training Framework: Based on verl framework, supports large-scale distributed training

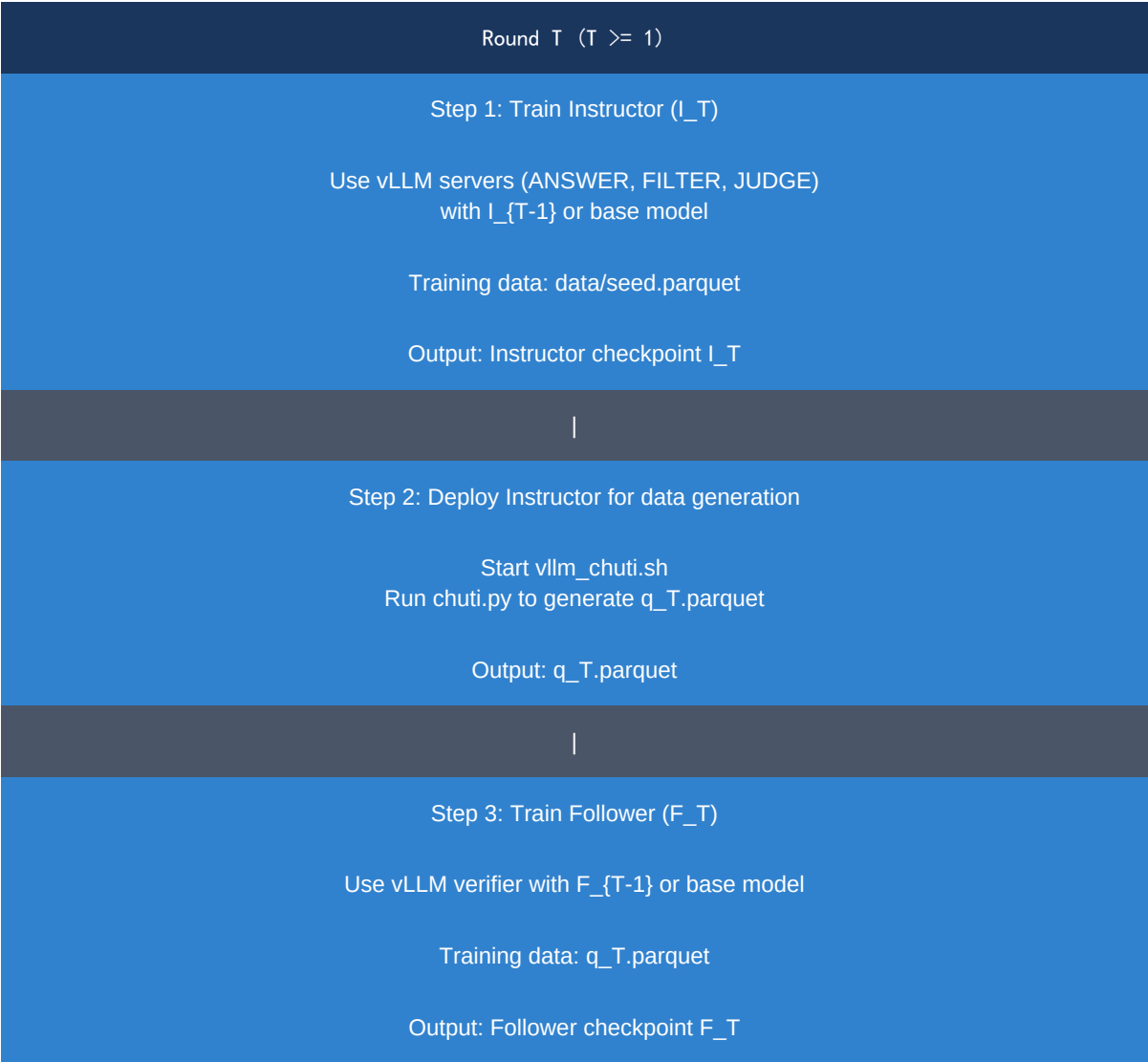
1.2 Key Files

File Path	Description
vllm_answer.sh	Answer generation model service (port 55222)
vllm_filter.sh	Constraint conflict filter (port 55333)
vllm_judge.sh	Constraint judgment model (port 55111)
vllm_verifier.sh	All-in-one verifier (port 55111)
examples/config.yaml	Instructor training configuration
examples/config1.yaml	Follower training configuration
examples/reward_function/instruction_reward.py	Reward function implementation
verl/utils/dataset.py	Dataset loading and processing
data/.../chuti.py	Constraint generation script

2. Iterative Training Workflow

The entire training process consists of multiple rounds, with each round containing three main steps: train Instructor, generate training data, train Follower.

2.1 Workflow Diagram



2.2 Key Design Principles

- Self-Improvement Loop: By using the latest Follower to evaluate Instructor, the Instructor is forced to generate harder instructions that the Follower cannot yet follow well.
- Progressive Difficulty: Each iteration produces more challenging multi-constraint instructions, driving continuous improvement of Follower capabilities.
- Flexible Constraint Types: Supports lexical, structural, format, and language style constraints.

3. Configuration Details

3.1 Instructor Training Config (config.yaml)

Instructor training uses GRPO algorithm on data/seed.parquet, with the goal of generating multi-constraint instructions that Follower can follow.

```
data: train_files: data/seed.parquet model: Qwen/Qwen2.5-7B-Instruct algorithm: adv_estimator:
grpo # GRPO algorithm kl_coef: 1.0e-2 online_filtering: false worker: rollout: temperature: 1.0
n: 5 # 5 rollouts per prompt tensor_parallel_size: 2 reward: reward_function:
./examples/reward_function/instruction_reward.py:instruction_compute_score
```

3.2 Follower Training Config (config1.yaml)

Follower training uses Instructor-generated data (q_Tk.parquet), with the goal of decomposing and correctly executing instructions with constraints.

```
data: train_files: data/Self-IF-Self-QWEN-7B/q_Tk.parquet model: Qwen/Qwen2.5-7B-Instruct
algorithm: adv_estimator: grpo kl_coef: 1.0e-2 worker: rollout: temperature: 1.0 n: 5
tensor_parallel_size: 2 reward: reward_type: batch reward_function:
./examples/reward_function/instruction_reward.py:instruction_compute_score trainer:
total_epochs: 3 (Turn 1) or 1 (Subsequent turns)
```

4. Reward Function Details

4.1 Instructor Reward Calculation

Instructor training uses a three-stage verification pipeline through vLLM model endpoints:

- Stage - Parse: Extract constraint list from Instructor-generated instructions
- Stage - Filter: Use FILTER_VLLM to detect logically conflicting constraints
- Stage - Generate Answer: Use ANSWER_VLLM to generate verification answers for each constraint
- Stage - Judge: Use JUDGE_VLLM to determine if Follower follows each constraint
- Stage - Score: Calculate compliance rate = followed constraints / total valid constraints

4.2 Follower Reward Calculation

Follower training uses instructions_registry to directly evaluate constraint compliance:

- * Parse instruction_id_list, kwargs, constraints from parquet ground_truth
- * Instantiate each constraint via instructions_registry.INSTRUCTION_DICT[ids]
- * Call build_description() to construct constraint description
- * Use check_following() to verify if Follower response satisfies constraints
- * Calculate score = followed constraints / total constraints

4.3 Supported Constraint Types

Category	Constraint Types	Example
Lexical	Keywords, Frequency, Forbidden Words, Letter Frequency	"Include beautiful" at least 3 times
Structural	Sentence Count, Paragraph Count, Bullet Point Count	Exactly 3 paragraphs, separated by ***
Format	Highlight, Title Wrapper, JSON Output	Wrap title with <<title>>
Language Style	Response Language, ALL CAPS, lowercase, No Comma	Output in English lowercase
Special Patterns	Repeat-Then-Answer, Exact Ending, Dual Response	Repeat question first, then answer

5. Detailed Usage Steps

5.1 Initial Setup

- * Prepare seed data: Ensure data/seed.parquet contains seed questions
- * Download base model: e.g., Qwen/Qwen2.5-7B-Instruct
- * Install dependencies: Ensure verl, vllm, ray are installed
- * Configure GPU resources: Each vLLM server requires specific GPU allocation

5.2 Round 1: Train Instructor

Step 1: Start vLLM evaluation servers

```
# Terminal 1: Answer model (port 55222) bash vllm_answer.sh # Terminal 2: Filter model (port 55333) bash vllm_filter.sh # Terminal 3: Judge model (port 55111) bash vllm_judge.sh
```

Step 2: Configure and run training

```
# Edit examples/qwen.sh MODEL_PATH=Qwen/Qwen2.5-7B-Instruct SAVE_PATH=/path/to/save/I_T1 # Start Instructor training bash examples/qwen.sh
```

5.3 Round 1: Train Follower

Step 1: Start Instructor service for data generation

```
# Start Instructor vLLM service bash data/Self-IF-Self-QWEN-7B/vllm_chuti.sh # Generate training data cd data/Self-IF-Self-QWEN-7B python chuti.py
```

Step 2: Configure and run Follower training

```
# Edit examples/qwen1.sh MODEL_PATH=Qwen/Qwen2.5-7B-Instruct SAVE_PATH=/path/to/save/F_T1 # Update config1.yaml: train_files -> q_T1.parquet # Start Follower training bash examples/qwen1.sh
```

5.4 Round T ($T \geq 2$): Iterative Training

From Round 2 onwards, use the previous round's Follower as the evaluation model.

```
# 1. Update vLLM servers to use F_{T-1} # Edit vllm_answer.sh, vllm_filter.sh, vllm_judge.sh
MODEL_PATH=/path/to/F_{T-1} # Restart all vLLM servers bash vllm_answer.sh bash vllm_filter.sh
bash vllm_judge.sh # 2. Train new Instructor I_T MODEL_PATH=/path/to/I_{T-1}
SAVE_PATH=/path/to/save/I_T bash examples/qwen.sh # 3. Generate new training data bash
data/Self-IF-Self-QWEN-7B/vllm_chuti.sh python chuti.py # Generate q_T.parquet # 4. Train new
Follower F_T # Update config1.yaml: train_files -> q_T.parquet MODEL_PATH=/path/to/F_{T-1}
SAVE_PATH=/path/to/save/F_T bash examples/qwen1.sh
```

6. Dataset Format

6.1 Seed Data Format (seed.parquet)

Seed data contains original questions for Instructor to learn constraint generation.

Field	Type	Description
prompt	string	Seed question text

6.2 Generated Data Format (q_Tk.parquet)

Multi-constraint instruction data generated by Instructor for Follower training.

Field	Type	Description
prompt	string	Complete instruction with constraints
seed_question	string	Original seed question
constraints	list[string]	Constraint list
constraint	list[string]	Constraint list (compatible field)
kwargs	list[dict]	Constraint parameters
instruction_id_list	list[string]	Constraint type ID list

6.3 Data Switching

The same code file supports two training modes, switched by commenting/uncommenting:

examples/reward_function/instruction_reward.py:

Instructor reward function (top, uncommented)

Follower reward function (commented)

verl/utils/dataset.py:

Instructor RLHFDataset (~line 220, uncommented)

Follower RLHFDataset (~lines 375-483, commented)

7. Quick Reference

7.1 Port Assignment

Port	Service	Purpose
55111	vllm_judge.sh / vllm_verifer.sh	Constraint judgment/verification
55222	vllm_answer.sh	Answer generation
55333	vllm_filter.sh	Conflict filtering

7.2 Training Round Summary

Round	Instructor	Follower	Training Data
T=1	I_1 (from base)	F_1 (from base)	q_T1.parquet
T=2	I_2 (eval by I_1, F_1)	F_2 (from F_1)	q_T2.parquet
T=3	I_3 (eval by I_2, F_2)	F_3 (from F_2)	q_T3.parquet
...	...	...	...

7.3 Command Quick Reference

Operation	Command
Start Instructor training	bash examples/qwen.sh
Start Follower training	bash examples/qwen1.sh
Start judge service	bash vllm_judge.sh
Start answer service	bash vllm_answer.sh
Start filter service	bash vllm_filter.sh
Generate training data	python data/Self-IF-Self-QWEN-7B/chuti.py